1. Test Plan

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

1.1 Test Objectives and Scope

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

Test Objectives

The objective of this testing activity is to verify that the selected core user workflows of the SmartHire application operate according to the current functional requirements and implementation.

The testing focuses on validating:

- Correct behavior for normal user workflows.
- Proper validation and handling of invalid inputs.
- Boundary and edge-case behavior.
- Persistence and state changes after user actions.
- Authentication and authorization behavior relevant to the selected workflows.
- Functional correctness of the AI-powered image-assisted job search feature.
- Correct interaction between the user interface and the implemented backend services.

The testing process will also identify defects, document failed test cases, support defect fixing, and verify fixes through retesting.

Test Scope

The PA5 manual functional testing scope consists of exactly five selected use cases:

1. **UC-AUTH-03 — Log In**
2. **UC-PROF-01 — Manage Candidate Profile**
3. **UC-JOB-01 — Browse, Search, and Filter Jobs**
4. **UC-APP-01 — Apply for a Job**
5. **Use Case / User Scenario — Feature 005 US2: Find Jobs from an Image Query**

Each selected use case contains 10 functional test cases, giving a total of **50 planned test cases**.

For PA5 testing, **Feature 005 US2 — Find Jobs from an Image Query** is treated as the fifth user-facing functional scenario/use case because it represents a complete Candidate workflow from image selection and consent through OCR/AI proposal review and job-search application. This wording is a PA5 test-document mapping and does not introduce a new official project use-case ID.

The test set includes:

- Positive test cases.
- Negative test cases.
- Boundary-value cases.
- Edge cases.

In Scope

The following testing activities are included:

- Manual functional testing through the current SmartHire user interface.
- Input validation.
- Navigation and workflow behavior.
- Authentication state relevant to selected use cases.
- Profile data persistence.
- Job searching, filtering, sorting, and pagination.
- Job application workflow and CV selection/upload.
- AI/OCR-assisted image job search.
- Functional error handling.
- Verification of visible results against expected results.
- Limited use of browser DevTools, database inspection, multiple browser sessions, and service logs when a result cannot be verified only through the UI.

Out of Scope

The following are outside the main scope of this PA5 manual functional test campaign:

- Full penetration testing.
- Comprehensive security auditing.
- Load and stress testing.
- Large-scale performance benchmarking.
- Full accessibility studies.
- Full usability studies.
- Testing unrelated features outside the five selected use cases.

- Re-execution of every automated unit, integration, component, or E2E test as part of the manual PA5 evidence.

Existing automated tests and Spec Kit artifacts may be used as references when reviewing and refining expected behavior, but manual PA5 execution results are recorded separately.

1.2 Features to Be Tested

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

The following application areas are selected for PA5 functional testing.

ID	Use Case	Main Functional Area	Planned Test Cases
UC-AUTH-03	Log In	Identity and Authentication	10
UC-PROF-01	Manage Candidate Profile	Candidate Profile Management	10
UC-JOB-01	Browse, Search, and Filter Jobs	Job Discovery	10
UC-APP-01	Apply for a Job	Job Application	10
Feature 005 US2	Find Jobs from an Image Query	OCR / AI-Assisted Job Search	10
Total			**50**

UC-AUTH-03 — Log In

Testing covers:

- Successful login.
- Required-field validation.
- Invalid email format.
- Incorrect credentials.
- Unverified account behavior.
- Suspended account behavior.
- Safe return navigation after login.
- Two-factor-authentication routing.
- Failed-login rate-limiting behavior.

UC-PROF-01 — Manage Candidate Profile

Testing covers:

- Opening the professional profile.
- Saving valid basic information.
- Persistence after reload.
- Skills.
- Work experience.
- Required-field validation.
- Phone-number validation.
- Education date validation.
- Social-link validation.
- Concurrent profile update handling.

UC-JOB-01 — Browse, Search, and Filter Jobs

Testing covers:

- Anonymous job browsing.
- Keyword search.
- Vietnamese text normalization.
- Location filtering.
- Combined filters.
- Sorting.
- Pagination.
- Empty search results.
- Salary boundaries.
- Exclusion of unavailable job postings.

UC-APP-01 — Apply for a Job

Testing covers:

- Authentication requirements.
- Applying with an existing Candidate CV.
- Uploading a CV during application.
- Required phone validation.

- Invalid phone validation.
- Required location validation.
- URL validation.
- Required CV validation.
- File-type validation.
- Application review confirmation.

Use Case / User Scenario — Feature 005 US2: Find Jobs from an Image Query

Testing covers:

- AI-processing consent.
- Processing a controlled image input.
- AI/OCR-generated job-search proposals.
- Applying generated criteria.
- Editing proposed criteria.
- Removing proposed criteria.
- Conflicts between manual and generated criteria.
- Unsupported image formats.
- Oversized images.
- Processing cancellation.
- Admission/rate-limit behavior.

A controlled synthetic image fixture with known visible information and matching seeded job data will be used when validating AI/OCR functional correctness.

1.3 Test Environment and Tools

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

Application Environment

Component	**Environment / Technology**
Application	SmartHire
Candidate Web URL	`http://localhost:3001`
Frontend / Server Framework	Next.js

Component	Environment / Technology
Backend	Node.js with Next.js App Router API routes
Database	PostgreSQL
ORM	Prisma
Authentication	Better Auth
Manual Test Browser	Google Chrome
Browser Debugging	Chrome DevTools
Container Environment	Docker / Docker Desktop where required
OCR / Image Processing	Project OCR/image-search services
AI Provider	Project-configured OpenAI integration
Existing Automated Test Technologies	Vitest and Playwright

Supporting Testing Tools

The following tools may be used during testing:

- Google Chrome for manual user interaction.
- Chrome DevTools for request/response inspection when required.
- PostgreSQL/Prisma database tools for preparing or verifying controlled test data.
- Separate browser profiles or incognito sessions for multi-session testing.
- Docker services required by OCR/image-search functionality.
- Application logs for troubleshooting failed cases.
- Existing Spec Kit specifications and automated tests as references for expected behavior.
- Git and GitHub for source history and defect-fix tracking.

Test Data

Testing will use controlled development/test data, including:

- Active verified Candidate accounts.
- Pending/unverified Candidate accounts.
- Suspended test accounts where required.
- A dedicated 2FA-enabled account.
- Candidate profiles containing controlled data.
- Active public job postings.

- Job postings with different locations, salaries, publication states, and deadlines.
- Jobs suitable for pagination and filtering tests.
- Valid synthetic CV files.
- Invalid file-type fixtures.
- Controlled image files for OCR/AI testing.
- Synthetic image-search posters with known visible truth values.
- Seeded jobs matching the image-search fixture.

Real user credentials, production data, secret authentication tokens, private API keys, and sensitive personal information must not be included in screenshots or test reports.

Execution Approach

Manual testing will be the authoritative PA5 execution method.

For every test case, the tester will:

1. Prepare the specified preconditions.
2. Perform the documented manual test steps.
3. Observe the actual application behavior.
4. Compare the actual behavior with the expected result.
5. Record the execution date.
6. Record the actual result.
7. Mark the case as Pass or Fail.
8. Create and link a bug report when a failure is found.

Automated test results may support investigation but will not be used as a substitute for manual execution evidence unless explicitly stated.

1.4 Test Schedule and Responsibilities

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

Testing will be executed in stages so that stable and low-dependency workflows are tested before environment-dependent workflows.

Phase	Activity	Responsible Person	Execution Period	Final Status
Phase 1	Review specifications and existing generated tests	Lưu Chí Hải with AI-assisted repository analysis	Pre-execution preparation; completed by 2026-08-21	Completed
Phase 2	Select five PA5 use cases/user scenarios	Lưu Chí Hải	Pre-execution preparation; completed by 2026-08-21	Completed
Phase 3	Review and refine 50 functional test cases	Lưu Chí Hải with AI-assisted repository analysis	Pre-execution preparation; completed by 2026-08-21	Completed
Phase 4	Prepare test environment and test data	Lưu Chí Hải / Nguyễn Minh Khôi / relevant development team members where necessary	Before and during the 2026-08-21 manual execution session	Completed
Phase 5	Execute authentication tests	Lưu Chí Hải	2026-08-21	Completed
Phase 6	Execute profile tests	Lưu Chí Hải	2026-08-21	Completed
Phase 7	Execute job-discovery tests	Nguyễn Minh Khôi	2026-08-21	Completed
Phase 8	Execute job-application tests	Nguyễn Minh Khôi	2026-08-21	Completed
Phase 9	Execute OCR/AI image-search tests	Lưu Chí Hải	2026-08-21	Completed
Phase 10	Record and investigate observed defects	Lưu Chí Hải for AUTH/IMG failures; relevant developers may investigate	2026-08-21	Completed
Phase 11	Retest failed cases after fixes become available	Relevant developer provides fix; assigned tester performs retest	After a fix becomes available; not yet scheduled	Not performed — reported defects remain Open
Phase 12	Prepare Test Summary and final merged test documentation	Lưu Chí Hải	2026-08-21	Completed

Recommended Execution Order

The planned execution order is:

1. Authentication.
2. Candidate Profile.
3. Job Browse/Search/Filter.
4. Job Application.
5. Image-Assisted Job Search.

Easy and stable test cases should be executed first.

Tests requiring special fixtures, multiple sessions, storage, TOTP, OCR workers, AI configuration, or controlled database state should be executed after the basic environment has been validated.

Responsibilities

Lưu Chí Hải

- Maintain and merge the PA5 Test Plan and final testing document.
- Execute and record the 10 AUTH test cases for **UC-AUTH-03 — Log In**.
- Execute and record the 10 PROF test cases for **UC-PROF-01 — Manage Candidate Profile**.
- Execute and record the 10 IMG test cases for **Feature 005 US2 — Find Jobs from an Image Query**.
- Determine Pass/Fail status for the cases he executed based on observed behavior.
- Document and link defects discovered during AUTH and IMG execution.
- Prepare the final Test Summary from the merged execution results.

Nguyễn Minh Khôi

- Execute and record the 10 JOB test cases for **UC-JOB-01 — Browse, Search, and Filter Jobs**.
- Execute and record the 10 APP test cases for **UC-APP-01 — Apply for a Job**.
- Determine Pass/Fail status for the cases he executed based on observed behavior.
- Document and link defects discovered in JOB or APP testing if any are observed.

Contribution Record

Test Area	Performed by	Reviewed by	Edited by
UC-AUTH-03 — Log In	Lưu Chí Hải	Nguyễn Gia Quốc Uy	Lưu Chí Hải

Test Area	Performed by	Reviewed by	Edited by
UC-PROF-01 — Manage Candidate Profile	Lưu Chí Hải	Nguyễn Gia Quốc Uy	Lưu Chí Hải
UC-JOB-01 — Browse, Search, and Filter Jobs	Nguyễn Minh Khôi	Nguyễn Gia Quốc Uy	Nguyễn Minh Khôi
UC-APP-01 — Apply for a Job	Nguyễn Minh Khôi	Lưu Chí Hải	Nguyễn Minh Khôi
Feature 005 US2 — Find Jobs from an Image Query	Lưu Chí Hải	Nguyễn Gia Quốc Uy	Lưu Chí Hải

Feature Developers

- Assist with test-data and environment preparation where necessary.
- Investigate defects related to their implemented features.
- Implement fixes when defects are confirmed.
- Provide technical clarification when the expected behavior is unclear.

AI Tools

AI tools may assist with:

- Repository analysis.
- Locating relevant implementation files.
- Reviewing generated test cases.
- Comparing test cases with current specifications and source code.
- Identifying missing edge cases.
- Preparing documentation.
- Explaining environment setup.
- Assisting developers with defect investigation.

AI tools must not fabricate execution results. Pass/Fail status and Actual Result fields must be based on actual execution against the running application.

AI assistance used during PA5 will be documented separately in the project's AI Usage Report.

1.5 Entry and Exit Criteria

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

Entry Criteria

Manual execution may begin when the following conditions are satisfied:

- The selected feature implementation is available in the current project version.
- The application can start successfully in the local development/testing environment.
- Required database migrations have been applied.
- PostgreSQL is available.
- Required test accounts have been prepared.
- Required test job postings and profile data have been prepared.
- Required test CV/image files are available.
- The selected test case has clear Preconditions, Steps, Test Data, and Expected Result.
- No blocking application startup error prevents the selected workflow from being exercised.

Additional entry conditions apply to environment-dependent tests:

Authentication / 2FA

- A dedicated TOTP-enabled test account exists before executing the 2FA test.

Profile Concurrency

- Two independent authenticated browser sessions can be created for the same test account.

Job Discovery

- Seed data required for filtering, sorting, pagination, salary, and lifecycle cases exists.

Job Application

- An active Candidate account exists.
- An active job accepting applications exists.
- Required CV storage is operational.
- Valid test CV files are available.

OCR / AI Image Search

Before image-assisted search cases are executed:

- Docker-dependent services required by the feature are operational.
- PostgreSQL is available.
- ClamAV/image validation dependencies are operational.

- OCR/image-search workers are operational.
- Required project AI configuration is available.
- Controlled synthetic image fixtures are prepared.
- Matching controlled job fixtures are available.

If these additional dependencies are unavailable, the affected test cases remain **Not Run** rather than being marked Pass or Fail without execution.

Exit Criteria

The PA5 functional-testing activity is considered complete when:

- All 50 selected functional test cases have been executed.
- Every executed case has an Execution Date.
- Every executed case has an Actual Result.
- Every executed case has a final Pass or Fail status.
- Every failed test case is linked to at least one bug report.
- Each bug report contains sufficient information to reproduce the defect.
- Confirmed defects have an appropriate status such as Open, Fixed, or Deferred.
- Fixed defects have been retested.
- Critical or high-severity unresolved defects are clearly reported before submission.
- Previously discovered bugs remain documented even when the final retest passes.
- Test execution totals have been calculated.
- Pass/Fail totals have been calculated for each tested feature/use case.
- The Test Summary has been completed.
- The final test documentation reflects the actual executed state of the application.

2. Test Cases and Execution Results

Performed by: Lưu Chí Hải & Nguyễn Minh Khôi | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

This section contains the 50 manual functional test cases selected for PA5. Each of the five selected use cases contains 10 test cases.

Execution-specific fields were initialized as **Not Run** or – during test design and were updated only after manual execution against the running SmartHire application. All 50 test cases below now contain their final execution date, actual result, Pass/Fail status, and Bug ID where applicable.

The following controlled local test-data aliases are used during execution. Temporary passwords are intentionally kept outside this report and are not committed to Git:

- `[ACTIVE_VERIFIED_CANDIDATE]` — `pa5.active.verified@smarthire.local`; Candidate state `ACTIVE`, email verified, 2FA disabled.
- `[UNVERIFIED_CANDIDATE]` — `pa5.unverified@smarthire.local`; supported unverified state `PENDING_VERIFICATION`.
- `[SUSPENDED_CANDIDATE]` — `pa5.suspended@smarthire.local`; Candidate state `SUSPENDED`, email verified.
- `[2FA_CANDIDATE]` — `pa5.twofactor@smarthire.local`; active and verified account reserved for supported TOTP enrollment before AUTH-09.
- `[RATE_LIMIT_CANDIDATE]` — `pa5.lockout@smarthire.local`; disposable active and verified Candidate reserved for AUTH-10.
- `[VALID_PASSWORD]` — the valid temporary password for the corresponding controlled account, stored outside this report.
- `[WRONG_PASSWORD]` — an intentionally incorrect password.
- `[VALID_CV_PDF]` — controlled valid synthetic CV file.
- `[INVALID_CV_FILE]` — controlled unsupported CV file.
- `[CONTROLLED_IMAGE_FIXTURE]` — approved synthetic image-search fixture with documented visible truth values.
- `[UNSUPPORTED_IMAGE]` — controlled unsupported image file.
- `[OVERSIZED_IMAGE]` — controlled image exceeding the configured upload-size limit.
- `[MAX_IMAGE_SIZE]` — `5,000,000` bytes (5 MB), the configured maximum accepted image upload size used for IMG-08.
- `[RATE_LIMIT_THRESHOLD]` — authenticated Candidate limit of 10 image-search admissions per rolling one-hour window, used for IMG-10.

2.0 Test Case Review and Refinement

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

The final 50-case PA5 set was not submitted unchanged from previously generated Spec Kit test artifacts. Existing generated tests were compared with the current specifications, implemented UI/backend behavior, and observable validation rules. Incorrect or vague assumptions were corrected, missing boundary and edge coverage was added, and AI-related tests were converted into controlled-input tests with explicit expected outcomes.

Concrete refinement examples include:

Refinement Area	What Was Reviewed / Refined	Final PA5 Evidence
Candidate profile required fields	An earlier assumption treated the Candidate headline as mandatory. Review of the current behavior showed that the headline is optional, so the test was replaced with a genuinely required-field check for Work Experience Job Title.	`PROF-06`
Login rate limiting	The rate-limit expectation was made specific instead of using a vague account-lockout assumption: five failed attempts are processed in the five-minute window and the sixth request is rejected with HTTP 429.	`AUTH-10`
Job discovery edge coverage	The final set includes explicit boundary/lifecycle checks rather than only normal search paths, including salary-boundary behavior and exclusion of future, expired, or closed jobs.	`JOB-09`, `JOB-10`
Job application validation	The final set includes negative validation and workflow-control cases such as unsupported CV type and mandatory final review confirmation.	`APP-09`, `APP-10`
AI/OCR functional correctness	Image-search correctness is evaluated with controlled synthetic images and known visible truth values instead of accepting unrestricted AI output as correct.	`IMG-02` to `IMG-06`
Image-search edge and boundary coverage	The final set explicitly covers unsupported file type, upload-size boundary, cancellation, and admission/rate-limit behavior.	`IMG-07` to `IMG-10`

These refinements demonstrate that the team reviewed and understood the generated testing material before manual execution. Manual observations remain authoritative for the recorded Pass/Fail results; expected results were not rewritten merely to make failed executions pass.

2.1 UC-AUTH-03 — Log In

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

AUTH-01 — Log in with valid credentials

Type: Positive

Preconditions:

- SmartHire is running.
- `[ACTIVE_VERIFIED_CANDIDATE]` exists and is not suspended.
- The account is not currently rate-limited.

Test Data:

- Email: `[ACTIVE_VERIFIED_CANDIDATE]`

- Password: `[VALID_PASSWORD]`

Steps:

1. Open `http://localhost:3001/login`.
2. Enter the valid Candidate email.
3. Enter the valid password.
4. Submit the login form.
5. Observe the resulting page.

Expected Result:

- Authentication succeeds.
- No credential error is displayed.
- The user is redirected away from `/login` to the authenticated Candidate area, expected to be `/dashboard` or the currently implemented authenticated landing page.

Execution Date: 2026-08-21 **Actual Result:** Login succeeded with valid verified Candidate credentials. The user was redirected to `http://localhost:3001/dashboard` and no authentication error was displayed. **Status:** Pass **Bug ID:** —

AUTH-02 — Submit login form without an email address

Type: Negative

Preconditions:

- SmartHire is running.
- The login page is accessible.

Test Data:

- Email: empty
- Password: `[VALID_PASSWORD]`

Steps:

1. Open `/login`.
2. Leave the email field empty.
3. Enter a password.
4. Submit the login form.
5. Observe validation behavior.

Expected Result:

- Login is not performed.
- The email field is identified as required.
- The user remains unauthenticated.

Execution Date: 2026-08-21 **Actual Result:** The login form was not submitted because the email field was empty. Required-field validation was displayed and the user remained unauthenticated. **Status:** Pass **Bug ID:** —

AUTH-03 — Submit login form without a password

Type: Negative

Preconditions:

- SmartHire is running.
- The login page is accessible.

Test Data:

- Email: `[ACTIVE_VERIFIED_CANDIDATE]`
- Password: empty

Steps:

1. Open `/login`.
2. Enter a valid Candidate email.
3. Leave the password field empty.
4. Submit the login form.
5. Observe validation behavior.

Expected Result:

- Login is not performed.
- The password field is identified as required.
- The user remains unauthenticated.

Execution Date: 2026-08-21

Actual Result: Login was blocked because the password field was empty. Required-field validation was displayed and no authenticated session was created.

Status: Pass

Bug ID: —

AUTH-04 — Enter a malformed email address

Type: Negative

Preconditions:

- SmartHire is running.
- The login page is accessible.

Test Data:

- Email: `invalid-email`
- Password: `[VALID_PASSWORD]`

Steps:

1. Open `/login`.
2. Enter `invalid-email` in the email field.
3. Enter a password.
4. Submit the login form.
5. Observe validation behavior.

Expected Result:

- Authentication is not attempted successfully.
- The malformed email is rejected or identified as invalid.
- The user remains on the login flow.

Execution Date: 2026-08-21

Actual Result: The malformed email address was rejected by the login form. Authentication did not complete and the user remained on the login flow.

Status: Pass

Bug ID: —

AUTH-05 — Log in with an incorrect password

Type: Negative

Preconditions:

- `[ACTIVE_VERIFIED_CANDIDATE]` exists.

- The account is not currently rate-limited before execution.

Test Data:

- Email: `[ACTIVE_VERIFIED_CANDIDATE]`
- Password: `[WRONG_PASSWORD]`

Steps:

1. Open `/login`.
2. Enter the valid Candidate email.
3. Enter an incorrect password.
4. Submit the form.
5. Observe the displayed response.

Expected Result:

- Login is denied.
- The user remains unauthenticated.
- The UI displays a generic authentication failure rather than exposing sensitive account details.
- If remaining-attempt or rate-limit information is implemented, it is updated consistently with the configured login rate-limit policy.

Execution Date: 2026-08-21

Actual Result: Login was denied for the valid Candidate email with an incorrect password. A generic authentication error was displayed and no authenticated session was created.

Status: Pass

Bug ID: —

AUTH-06 — Attempt login with an unverified account

Type: Negative

Preconditions:

- `[UNVERIFIED_CANDIDATE]` exists.
- The account has valid credentials but has not completed required email verification.

Test Data:

- Email: `[UNVERIFIED_CANDIDATE]`
- Password: `[VALID_PASSWORD]`

Steps:

1. Open `/login`.
2. Enter the unverified account email.
3. Enter its valid password.
4. Submit the login form.
5. Observe the result.

Expected Result:

- Access to the normal authenticated Candidate area is denied.
- The application indicates that account/email verification is required or routes the user into the supported verification flow.
- The account is not treated as a normally verified Candidate.

Execution Date: 2026-08-21

Actual Result: Login was denied for the pending-verification Candidate account and no authenticated session was created. However, the application displayed the same generic "Email or password is incorrect." response used for incorrect credentials or a non-existent email. No verification-specific message, verification link, or redirect to the email-verification flow was provided.

Status: Fail

Bug ID: BUG-AUTH-06

AUTH-07 — Attempt login with a suspended account

Type: Negative

Preconditions:

- `[SUSPENDED_CANDIDATE]` exists and is currently suspended.

Test Data:

- Email: `[SUSPENDED_CANDIDATE]`
- Password: `[VALID_PASSWORD]`

Steps:

1. Open `/login`.
2. Enter the suspended account email.
3. Enter the correct password.

4. Submit the form.
5. Observe the resulting behavior.

Expected Result:

- Normal Candidate access is denied.
- The user is not granted an authenticated Candidate session that bypasses the suspension.
- Appropriate suspended-account handling is displayed.

Execution Date: 2026-08-21

Actual Result: Login with the suspended Candidate's correct credentials did not grant normal Candidate access. The application displayed suspended-account handling and no usable authenticated Candidate session was created.

Status: Pass

Bug ID: —

AUTH-08 — Return to a protected page after successful login

Type: Positive

Preconditions:

- The tester is signed out.
- `[ACTIVE_VERIFIED_CANDIDATE]` exists.
- `/profile` requires authentication.

Test Data:

- Target route: `/profile`
- Valid Candidate credentials.

Steps:

1. While signed out, navigate directly to `http://localhost:3001/profile`.
2. Verify that the application redirects to the login flow.
3. Enter valid credentials.
4. Complete login.
5. Observe the post-login destination.

Expected Result:

- The protected route cannot be accessed while signed out.

- After successful login, the application safely returns the user to `/profile` or the supported equivalent return destination.
- The return destination is not replaced by an unsafe external URL.

Execution Date: 2026-08-21

Actual Result: While signed out, navigating to `/profile` redirected the user to the login flow. After successful authentication, the application redirected the user to `/dashboard` instead of returning to the originally requested `/profile` route, even though `/profile` is a valid internal return destination.

Status: Fail

Bug ID: BUG-AUTH-08

AUTH-09 — Route a 2FA-enabled account to the two-factor challenge

Type: Positive

Preconditions:

- `[2FA_CANDIDATE]` exists.
- Two-factor authentication is enabled for this account.
- The account is active and verified.

Test Data:

- Email: `[2FA_CANDIDATE]`
- Password: `[VALID_PASSWORD]`

Steps:

1. Open `/login`.
2. Enter the 2FA-enabled Candidate email.
3. Enter the correct password.
4. Submit the login form.
5. Observe the next authentication step.

Expected Result:

- Password authentication alone does not immediately complete the login.
- The user is routed to the implemented two-factor authentication challenge.
- The protected Candidate area is not accessible until the second factor is successfully completed.

Execution Date: 2026-08-21

Actual Result: Correct password authentication for the 2FA-enabled Candidate routed the user to the two-factor challenge instead of completing login immediately. Protected Candidate access was granted only after the valid TOTP code was entered.

Status: Pass

Bug ID: —

AUTH-10 — Enforce failed-login rate limit

Type: Boundary

Preconditions:

- `[RATE_LIMIT_CANDIDATE]` is a disposable active and verified Candidate account.
- The account has not been used for login testing within the current five-minute rate-limit window.
- The configured login rate limit is five attempts per five-minute window.

Test Data:

- Email: `[RATE_LIMIT_CANDIDATE]`
- Password: `[WRONG_PASSWORD]`

Steps:

1. Open `/login`.
2. Enter the disposable Candidate email and an incorrect password.
3. Submit five failed login attempts within the same five-minute window.
4. Observe the response after each attempt.
5. Submit a sixth login attempt within the same five-minute window.
6. Observe the sixth response.

Expected Result:

- The first five attempts are processed according to the normal invalid-credentials behavior.
- The sixth attempt within the same five-minute window is rejected by the login rate limiter.
- The response represents HTTP 429 Too Many Requests.
- The Candidate remains unauthenticated.
- The rate limit resets after the fixed five-minute window expires.

Execution Date: 2026-08-21

Actual Result: Five failed login requests were processed within the same five-minute window. The sixth request was rejected by the login rate limiter with HTTP 429 Too Many Requests. The Candidate remained unauthenticated.

Status: Pass

Bug ID: —

2.2 UC-PROF-01 — Manage Candidate Profile

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

PROF-01 — Open the Candidate profile

Type: Positive

Preconditions:

- The tester is authenticated as `[ACTIVE_VERIFIED_CANDIDATE]`.

Test Data:

- Route: `/profile`

Steps:

1. Log in as the active Candidate.
2. Navigate to `/profile`.
3. Wait for profile data to load.
4. Observe the displayed profile information.

Expected Result:

- The Candidate profile page loads successfully.
- The authenticated Candidate's profile data is displayed.
- No other user's private profile data is displayed.

Execution Date: 2026-08-21

Actual Result: The authenticated Candidate profile loaded successfully and displayed the Candidate's profile information without an application error.

Status: Pass

Bug ID: —

PROF-02 — Save valid basic profile information

Type: Positive

Preconditions:

- The tester is authenticated.
- The Candidate profile page is accessible.

Test Data:

- Headline: `Software Engineering Student`
- Phone: a valid test phone number supported by the application.
- Location: a valid test location.

Steps:

1. Open `/profile`.
2. Edit the Candidate's basic profile information.
3. Enter valid values in the editable fields.
4. Save the changes.
5. Observe the save result.

Expected Result:

- The profile update succeeds.
- A success state or updated profile is displayed.
- The newly entered values are visible after saving.

Execution Date: 2026-08-21

Actual Result: The valid profile information was saved successfully. The updated headline, phone number, and location were displayed with the newly entered values after saving.

Status: Pass

Bug ID: —

PROF-03 — Verify profile persistence after reload

Type: Positive

Preconditions:

- PROF-02 has successfully saved known test values.

Test Data:

- The values saved during PROF-02.

Steps:

1. Confirm the updated profile values are visible.
2. Reload the browser page.
3. Wait for the profile to load again.
4. Compare the displayed values with the previously saved values.

Expected Result:

- The profile update persists after page reload.
- The saved values are retrieved from persistent application state rather than reverting to the previous values.

Execution Date: 2026-08-21

Actual Result: After reloading the profile page, the values saved in the previous test remained unchanged and were loaded correctly from persistent application state.

Status: Pass

Bug ID: —

PROF-04 — Add skills to the Candidate profile

Type: Positive

Preconditions:

- The tester is authenticated.
- The profile skills area is available.

Test Data:

- Skills: `TypeScript`, `PostgreSQL`

Steps:

1. Open the profile skills section.
2. Add `TypeScript`.
3. Add `PostgreSQL`.
4. Save the profile.
5. Reload or reopen the skills section.

Expected Result:

- Both valid skills are accepted.
- The skills appear on the Candidate profile after saving.
- The saved skill data persists when the profile is reopened.

Execution Date: 2026-08-21

Actual Result: The `TypeScript` and `PostgreSQL` skills were accepted and saved successfully. Both skills remained visible after the skills section was reopened.

Status: Pass

Bug ID: —

PROF-05 — Add valid work experience

Type: Positive

Preconditions:

- The tester is authenticated.
- The work-experience editor is available.

Test Data:

- Position: `Software Engineering Intern`
- Company: `PA5 Test Company`
- Start date: valid past date.
- End date: valid date after the start date.

Steps:

1. Open the work-experience section.
2. Add a new experience entry.
3. Enter valid company, position, and date information.
4. Save the entry.
5. Observe the resulting profile.

Expected Result:

- The valid work-experience entry is accepted.
- The new experience appears in the Candidate profile.
- The saved entry persists.

Execution Date: 2026-08-21

Actual Result: The valid work-experience entry was saved successfully and appeared on the Candidate profile with the entered position, company, and date information. The entry remained available after reopening the profile.

Status: Pass

Bug ID: —

PROF-06 — Reject work experience with missing required job title

Type: Negative

Preconditions:

- The tester is authenticated.
- The work-experience editor is available.

Test Data:

- Job Title: empty.
- Company: `PA5 Test Company`
- Start date: a valid past date.
- End date: a valid date after the start date.

Steps:

1. Open the work-experience section.
2. Add a new work-experience entry.
3. Leave the required Job Title field empty.
4. Enter valid values for the remaining required fields.
5. Attempt to save the work-experience entry.
6. Observe the validation behavior.

Expected Result:

- The invalid work-experience entry is rejected.
- Validation feedback identifies the missing required Job Title.
- The invalid experience entry is not persisted in the Candidate profile.

Execution Date: 2026-08-21

Actual Result: The work-experience entry was rejected when the required Job Title field was left empty. Validation feedback was displayed for the missing field, and the invalid experience entry was not saved to the Candidate profile.

Status: Pass

Bug ID: —

PROF-07 — Enter an invalid phone number

Type: Negative

Preconditions:

- The tester is authenticated.
- The phone field is editable.

Test Data:

- Phone: `abc-invalid-phone`

Steps:

1. Open the profile editor.
2. Enter the invalid phone value.
3. Attempt to save the profile.
4. Observe the result.

Expected Result:

- The invalid phone number is rejected.
- Appropriate validation is displayed.
- The invalid value is not persisted as a valid Candidate phone number.

Execution Date: 2026-08-21

Actual Result: The invalid phone value `abc-invalid-phone` was rejected and validation feedback was displayed. The invalid value was not persisted as the Candidate's phone number.

Status: Pass

Bug ID: —

PROF-08 — Enter an invalid education date range

Type: Negative

Preconditions:

- The tester is authenticated.
- The education editor is available.

Test Data:

- Start date: later date.

- End date: earlier date.

Steps:

1. Add or edit an education entry.
2. Set the start date later than the end date.
3. Attempt to save the entry.
4. Observe validation behavior.

Expected Result:

- The invalid chronological date range is rejected.
- The user receives appropriate validation.
- The inconsistent education record is not persisted.

Execution Date: 2026-08-21

Actual Result: The education entry was rejected because the start date was later than the end date. Appropriate validation feedback was displayed and the invalid education record was not saved.

Status: Pass

Bug ID: —

PROF-09 — Enter an invalid social-profile URL

Type: Negative

Preconditions:

- The tester is authenticated.
- A social/profile URL field is available.

Test Data:

- URL: `not-a-valid-url`

Steps:

1. Open the relevant profile section.
2. Enter the invalid URL.
3. Attempt to save the profile.
4. Observe validation behavior.

Expected Result:

- The malformed URL is rejected.
- The user receives URL validation feedback.
- The invalid URL is not persisted as a valid social/profile link.

Execution Date: 2026-08-21

Actual Result: The malformed social-profile URL `not-a-valid-url` was rejected and URL validation feedback was displayed. The invalid URL was not persisted in the Candidate profile.

Status: Pass

Bug ID: —

PROF-10 — Handle concurrent profile updates

Type: Edge

Preconditions:

- Two independent browser sessions are available.
- Both sessions are authenticated as the same controlled Candidate account.

Test Data:

- Session A headline: `Profile Update A`
- Session B headline: `Profile Update B`

Steps:

1. Open the same Candidate profile in Session A and Session B.
2. In Session A, change the headline to `Profile Update A` and save.
3. Without refreshing Session B, change the headline there to `Profile Update B`.
4. Attempt to save Session B.
5. Observe the conflict-handling behavior.
6. Reload the final profile.

Expected Result:

- The application detects or safely handles the stale concurrent update.
- A stale session must not silently overwrite a newer update without the application's intended conflict handling.
- The final persisted state is deterministic and consistent with the implemented concurrency policy.

Execution Date: 2026-08-21

Actual Result: The application detected that another session had saved a newer profile revision. The second valid update was saved and replaced the newer revision, while the application explicitly displayed the message: "Saved. Another session had a newer profile revision, so this valid update replaced it." The concurrent update was therefore handled explicitly rather than silently.

Status: Pass

Bug ID: —

2.3 UC-JOB-01 — Browse, Search, and Filter Jobs

Performed by: Nguyễn Minh Khôi | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Nguyễn Minh Khôi

JOB-01 — Browse jobs while signed out

Type: Positive

Preconditions:

- Seeded active public jobs exist.
- The tester is signed out.

Test Data:

- Route: `/jobs`

Steps:

1. Open `http://localhost:3001/jobs` while signed out.
2. Wait for the page to load.
3. Observe the displayed job listings.
4. Open one public job listing if available.

Expected Result:

- The public job board is accessible without authentication.
- Active public jobs are displayed.
- Public job details can be viewed according to the current job-discovery workflow.

Execution Date: 2026-08-21

Actual Result: The public job board loaded successfully without requiring authentication. Active public jobs were displayed in the listing, and opening a job showed its public details correctly.

Status: Pass

Bug ID: —

JOB-02 — Search jobs by keyword

Type: Positive

Preconditions:

- At least one seeded job contains a known searchable keyword.

Test Data:

- Keyword: a known keyword from the seeded job dataset, such as a known job title or skill.

Steps:

1. Open `/jobs`.
2. Enter the selected known keyword in the search field.
3. Submit or apply the search.
4. Observe the result list.

Expected Result:

- Returned jobs match the entered keyword according to the implemented search behavior.
- Unrelated jobs are not presented as matching results without a supported match reason.
- The active search criterion remains visible to the user.

Execution Date: 2026-08-21

Actual Result: Search returned only jobs matching the entered keyword. No unrelated jobs appeared in the results, and the active search term remained visible in the UI after the search executed.

Status: Pass

Bug ID: —

JOB-03 — Search Vietnamese text with diacritic/case normalization

Type: Edge

Preconditions:

- Seed data contains a job with controlled Vietnamese searchable text.

Test Data:

- Known Vietnamese job/location term.
- Alternate case and/or diacritic form of that term.

Steps:

1. Identify a seeded job containing the controlled Vietnamese term.
2. Search using the original form and note the matching result.
3. Search using a case-varied and/or supported diacritic-normalized form.
4. Compare the returned results.

Expected Result:

- Search normalization behaves consistently with the implemented Vietnamese-search requirements.
- The controlled matching job remains discoverable when using the supported normalized form.

Execution Date: 2026-08-21

Actual Result: The controlled job was returned consistently for both the original Vietnamese term and its case-varied/diacritic-normalized form. Normalization behavior matched the implemented requirements.

Status: Pass

Bug ID: —

JOB-04 — Filter jobs by location

Type: Positive

Preconditions:

- Seeded jobs exist in multiple locations.

Test Data:

- A known location containing at least one active job.

Steps:

1. Open `/jobs`.
2. Select or enter the controlled location filter.
3. Apply the filter.
4. Inspect the returned job listings.

Expected Result:

- Returned jobs satisfy the selected location criterion according to the implemented location-filter rules.
- Jobs that clearly do not match the selected location are excluded unless supported remote/nationwide behavior applies.

Execution Date: 2026-08-21

Actual Result: Applying the location filter returned only jobs matching the selected location (plus supported remote/nationwide jobs where applicable). Non-matching jobs were correctly excluded.

Status: Pass

Bug ID: —

JOB-05 — Apply multiple job filters together

Type: Positive

Preconditions:

- Seed data contains at least one job matching a known combination of filters.

Test Data:

- Controlled keyword.
- Controlled location.
- One additional supported filter.

Steps:

1. Open `/jobs`.
2. Enter the controlled keyword.
3. Apply the controlled location.
4. Apply one additional supported filter.
5. Execute the combined search.
6. Inspect the results.

Expected Result:

- Returned jobs satisfy the combined active criteria.
- The active criteria remain represented in the UI.
- Removing or changing one criterion updates the search appropriately.

Execution Date: 2026-08-21

Actual Result: The combined keyword, location, and additional filter returned results satisfying all three criteria simultaneously. All active filters remained visible in the UI, and removing one criterion correctly updated the result set.

Status: Pass

Bug ID: —

JOB-06 — Sort job-search results

Type: Positive

Preconditions:

- Multiple matching jobs exist.
- At least one sorting option is available.

Test Data:

- A supported sort option such as the current newest/relevance/salary option.

Steps:

1. Open `/jobs`.
2. Produce a result set containing multiple jobs.
3. Select a supported sort option.
4. Observe the ordering of results.
5. If practical, compare several displayed values with the chosen sort rule.

Expected Result:

- The result list is reordered according to the selected supported sort option.
- The selected sort option remains active and visible.
- Sorting does not unexpectedly clear unrelated active search criteria.

Execution Date: 2026-08-21

Actual Result: Selecting a sort option correctly reordered the result list. The chosen sort option stayed visibly active, and unrelated active search criteria were preserved after sorting.

Status: Pass

Bug ID: —

JOB-07 — Preserve search criteria across pagination

Type: Edge

Preconditions:

- The selected search produces enough results for more than one page.

Test Data:

- Controlled search/filter criteria that return multiple pages.

Steps:

1. Open `/jobs`.
2. Apply the controlled criteria.
3. Navigate to the next page of results.
4. Observe the active criteria and result set.
5. Navigate back to the previous page if supported.

Expected Result:

- Pagination changes the result page without silently discarding active search/filter criteria.
- The URL/UI state remains consistent with the current search.
- Results on subsequent pages continue to satisfy the active criteria.

Execution Date: 2026-08-21

Actual Result: Navigating to subsequent pages retained the active search/filter criteria, and the URL/UI state stayed consistent. Results on later pages continued to satisfy the applied criteria. Navigating back to the previous page also worked as expected.

Status: Pass

Bug ID: —

JOB-08 — Handle a search with no matching jobs

Type: Edge

Preconditions:

- The job board is available.

Test Data:

- Keyword: `zzzz-no-matching-pa5-job-987654`

Steps:

1. Open `/jobs`.
2. Search for the intentionally nonmatching keyword.
3. Observe the result state.
4. Clear the search criterion.
5. Observe whether normal job results recover.

Expected Result:

- The application displays a valid empty/no-results state rather than an application error.
- The user can clear or modify the search.
- Normal job listings become available again after removing the impossible criterion.

Execution Date: 2026-08-21

Actual Result: Searching the nonmatching keyword produced a clean empty/no-results state with no application error. Clearing the search criterion restored the normal job listings correctly.

Status: Pass

Bug ID: —

JOB-09 — Test salary-filter boundary behavior

Type: Boundary

Preconditions:

- A controlled seeded active job has a known salary boundary value `[S]`.
- Salary filtering is available.

Test Data:

- Known salary value: `[S]`
- Boundary values based on `[S]`.

Steps:

1. Identify a seeded active job with known salary `[S]`.
2. Apply a salary criterion whose boundary is exactly `[S]`.
3. Verify whether the controlled job is included according to the specified inclusive/exclusive rule.
4. Adjust the boundary just beyond `[S]`.
5. Observe whether the controlled job is included or excluded consistently.

Expected Result:

- Salary filtering handles the exact boundary according to the defined implementation rule.
- Moving beyond the boundary changes eligibility consistently.
- No off-by-one or incorrect comparison behavior is observed.

Execution Date: 2026-08-21

Actual Result: The exact boundary value `[S]` was handled consistently with the defined inclusive/exclusive rule, and shifting the boundary past `[S]` correctly changed eligibility. No off-by-one or comparison errors

were observed.

Status: Pass

Bug ID: —

JOB-10 — Exclude future, expired, or closed jobs from public discovery

Type: Edge

Preconditions:

- Controlled job fixtures include active jobs and at least one unavailable lifecycle state such as future, expired, or closed.

Test Data:

- Known active job.
- Known future/expired/closed job fixtures.

Steps:

1. Open `/jobs`.
2. Search for or otherwise locate the controlled active job.
3. Verify that the active job can be discovered.
4. Search for controlled future, expired, or closed fixtures using known identifying data.
5. Observe whether unavailable jobs appear in normal public discovery.

Expected Result:

- Eligible active public jobs are discoverable.
- Future, expired, closed, or otherwise unavailable postings are excluded from normal public job discovery according to the implemented lifecycle rules.

Execution Date: 2026-08-21

Actual Result: The known active job was discoverable as expected, while the future, expired, and closed job fixtures did not appear in normal public job discovery, consistent with the implemented lifecycle rules.

Status: Pass

Bug ID: —

2.4 UC-APP-01 — Apply for a Job

Performed by: Nguyễn Minh Khôi | *Reviewed by:* Lưu Chí Hải | *Edited by:* Nguyễn Minh Khôi

APP-01 — Require login before applying for a job

Type: Edge

Preconditions:

- The tester is signed out.
- A seeded active job accepting applications exists.

Test Data:

- Active job application URL.
- Valid Candidate credentials.

Steps:

1. While signed out, open the application route for an active job.
2. Attempt to begin the application flow.
3. Observe the authentication handling.
4. Log in with valid Candidate credentials.
5. Observe the destination after authentication.

Expected Result:

- A signed-out user cannot submit a Candidate application.
- The application requires authentication.
- After successful login, the user is returned to the intended application flow or supported equivalent destination.

Execution Date: 2026-08-21

Actual Result: The signed-out user was blocked from submitting an application and was prompted to authenticate. After logging in with valid Candidate credentials, the user was correctly returned to the intended application flow.

Status: Pass

Bug ID: —

APP-02 — Apply using an existing Candidate CV

Type: Positive

Preconditions:

- The Candidate is authenticated.

- The Candidate already has a valid stored CV.
- An active job accepts applications.

Test Data:

- Existing Candidate CV.
- Active job.

Steps:

1. Open the active job's application flow.
2. Complete valid required Candidate/contact information.
3. Select the existing Candidate CV.
4. Continue through the application flow.
5. Complete the required review/confirmation step.
6. Submit the application.

Expected Result:

- The existing valid CV can be selected.
- The application passes CV validation.
- A valid application can be submitted successfully when all other required information is complete.

Execution Date: 2026-08-21

Actual Result: The existing stored CV was selected successfully and passed validation. With all other required information complete, the application was submitted successfully.

Status: Pass

Bug ID: —

APP-03 — Upload a valid CV during the application flow

Type: Positive

Preconditions:

- The Candidate is authenticated.
- CV upload/storage dependencies are operational.
- An active job accepts applications.

Test Data:

- `[VALID_CV_PDF]`

Steps:

1. Open the active job application flow.
2. Choose the supported option to upload a CV.
3. Select `[VALID_CV_PDF]`.
4. Wait for upload/validation processing.
5. Complete the other required application information.
6. Proceed through review and submit if enabled.

Expected Result:

- The valid CV file is accepted.
- The uploaded CV becomes available to the application workflow.
- The Candidate can proceed with the application using the uploaded CV.

Execution Date: 2026-08-21

Actual Result: The valid CV file uploaded and validated successfully, and became available for use in the application workflow. The Candidate was able to proceed and complete the application using the uploaded CV.

Status: Pass

Bug ID: —

APP-04 — Attempt application without required phone number

Type: Negative

Preconditions:

- The Candidate is authenticated.
- An active job accepts applications.

Test Data:

- Phone: empty.
- Other required fields: valid.

Steps:

1. Open the application flow.
2. Complete all required information except phone number.

3. Leave the required phone field empty.
4. Attempt to continue or submit.
5. Observe validation behavior.

Expected Result:

- The workflow does not accept the incomplete application.
- The required phone field is identified.
- The application is not submitted as complete.

Execution Date: 2026-08-21

Actual Result: Submission was blocked with the empty phone field, and the required phone field was clearly flagged to the user. The application was not submitted.

Status: Pass

Bug ID: —

APP-05 — Enter an invalid phone number

Type: Negative

Preconditions:

- The Candidate is authenticated.
- An active job accepts applications.

Test Data:

- Phone: `abc-invalid-phone`

Steps:

1. Open the application flow.
2. Complete required fields with valid data.
3. Enter the invalid phone value.
4. Attempt to continue or submit.
5. Observe validation behavior.

Expected Result:

- The malformed phone number is rejected.
- The Candidate receives validation feedback.

- The invalid application is not submitted successfully.

Execution Date: 2026-08-21

Actual Result: The malformed phone value `abc-invalid-phone` was rejected with clear validation feedback shown to the Candidate, and the application was not submitted.

Status: Pass

Bug ID: —

APP-06 — Attempt application without required location

Type: Negative

Preconditions:

- The Candidate is authenticated.
- Location is required by the current application workflow.

Test Data:

- Location: empty.
- Other required values: valid.

Steps:

1. Open the application flow.
2. Complete all other required values.
3. Leave the required location field empty.
4. Attempt to continue or submit.
5. Observe validation behavior.

Expected Result:

- The incomplete application is rejected.
- The required location field is identified.
- The application is not submitted as valid.

Execution Date: 2026-08-21

Actual Result: The application was rejected with the location field empty, and the required location field was clearly identified to the Candidate. The application was not submitted.

Status: Pass

Bug ID: —

APP-07 — Enter an invalid LinkedIn or portfolio URL

Type: Negative

Preconditions:

- The Candidate is authenticated.
- The application workflow exposes a LinkedIn or portfolio URL field.

Test Data:

- URL: `not-a-valid-url`

Steps:

1. Open the application flow.
2. Complete required fields with valid values.
3. Enter the malformed URL.
4. Attempt to continue or submit.
5. Observe the result.

Expected Result:

- The malformed URL is rejected according to the application's URL validation rules.
- The Candidate receives validation feedback.
- The invalid value does not result in a successful application submission.

Execution Date: 2026-08-21

Actual Result: The malformed URL `not-a-valid-url` was rejected by the URL validation rules, with clear feedback shown to the Candidate. No successful submission occurred with the invalid value.

Status: Pass

Bug ID: —

APP-08 — Attempt application without a CV

Type: Negative

Preconditions:

- The Candidate is authenticated.
- A CV is required for the target application flow.
- An active job accepts applications.

Test Data:

- CV: none.

Steps:

1. Open the active job application flow.
2. Complete other required Candidate information.
3. Do not select an existing CV.
4. Do not upload a new CV.
5. Attempt to proceed or submit.

Expected Result:

- The application cannot be completed without the required CV.
- The missing CV requirement is communicated to the Candidate.
- No valid application is created without the required CV.

Execution Date: 2026-08-21

Actual Result: The application could not be completed without a CV; the missing requirement was clearly communicated to the Candidate, and no application record was created.

Status: Pass

Bug ID: —

APP-09 — Upload an unsupported CV file type

Type: Negative

Preconditions:

- The Candidate is authenticated.
- CV upload functionality is available.

Test Data:

- `[INVALID_CV_FILE]` with a file type not permitted for Candidate CV upload.

Steps:

1. Open the application CV upload step.
2. Attempt to select/upload `[INVALID_CV_FILE]`.
3. Observe client-side and/or server-side validation.

4. Attempt to continue if the UI permits it.

Expected Result:

- The unsupported CV file is rejected.
- The invalid file is not accepted as a valid Candidate CV.
- The Candidate receives an appropriate validation/error response.
- The invalid upload does not allow successful application submission.

Execution Date: 2026-08-21

Actual Result: The unsupported file type was rejected by validation and was not accepted as a valid CV. The Candidate received an appropriate error message, and no successful submission occurred with the invalid upload.

Status: Pass

Bug ID: —

APP-10 — Require review confirmation before final submission

Type: Negative

Preconditions:

- The Candidate is authenticated.
- All application data before the review step is valid.
- A valid CV is available.

Test Data:

- Valid Candidate information.
- Valid CV.

Steps:

1. Complete the application flow with valid data.
2. Reach the final review/confirmation step.
3. Leave the required review/confirmation control unconfirmed.
4. Attempt final submission.
5. Observe the result.
6. Complete the required confirmation and verify that the workflow can then proceed.

Expected Result:

- Final application submission is blocked while mandatory review/confirmation is incomplete.
- The user is informed that confirmation is required.
- Completing the confirmation enables valid final submission.

Execution Date: 2026-08-21

Actual Result: Final submission was blocked while the confirmation control was left unconfirmed, and the user was informed that confirmation was required. Completing the confirmation then allowed the application to be submitted successfully.

Status: Pass

Bug ID: —

2.5 Use Case / User Scenario — Feature 005 US2: Find Jobs from an Image Query

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

These test cases were executed after the required image-search test environment and controlled fixtures were prepared. During execution, the live OCR path repeatedly returned `OCR_UNAVAILABLE` for IMG-02 through IMG-06; those observations are recorded as failed test cases and linked to `BUG-IMG-02`. Pre-OCR validation, cancellation, and admission-control cases remained executable and were recorded independently.

Controlled synthetic fixtures with documented visible truth values were used so that OCR/AI output could be evaluated against known evidence rather than against an unrestricted or arbitrary model response.

IMG-01 — Require consent before image processing

Type: Positive

Preconditions:

- Image-search UI is available.
- Required image-search services are operational.

Test Data:

- `[CONTROLLED_IMAGE_FIXTURE]`

Steps:

1. Open the image-assisted job-search interface.
2. Do not provide the required image-processing consent.
3. Attempt to select or process an image.
4. Observe whether processing is gated.

5. Provide the required consent.
6. Observe whether image selection/processing becomes available.

Expected Result:

- Image processing cannot proceed before required consent.
- Providing valid consent enables the supported image-selection/processing workflow.
- Consent state is respected by the feature.

Execution Date: 2026-08-21

Actual Result: Image processing was unavailable before the required consent was provided. After consent was granted, the application allowed the image-selection and processing workflow to continue.

Status: Pass

Bug ID: —

IMG-02 — Process a controlled synthetic job poster

Type: Positive

Preconditions:

- Image-search services are operational.
- `[CONTROLLED_IMAGE_FIXTURE]` has documented visible truth values.
- Matching controlled job data exists.

Test Data:

- `[CONTROLLED_IMAGE_FIXTURE]`
- Documented truth set for visible job-search information.

Steps:

1. Open the image-assisted search interface.
2. Provide required consent.
3. Select `[CONTROLLED_IMAGE_FIXTURE]`.
4. Start image processing.
5. Wait for OCR/AI processing to complete.
6. Review the proposed search criteria.

Expected Result:

- The supported image is accepted and processed.
- Processing completes without an unexpected application error.
- Generated proposals are based on information supported by the visible controlled fixture.
- The proposals are presented for user review rather than silently becoming irreversible search criteria.

Execution Date: 2026-08-21

Actual Result: The controlled supported job-poster image was admitted successfully, passed malware scanning and image normalization, but repeatedly failed during the OCR stage with the persisted error code `OCR_UNAVAILABLE`. No OCR text or `TypeScript` proposal was produced. The application displayed "Image search could not continue" and allowed the user to continue with ordinary manual search.

Status: Fail

Bug ID: BUG-IMG-02

IMG-03 — Apply reviewed image-generated search proposals

Type: Positive

Preconditions:

- IMG-02 produces at least one valid reviewed proposal.
- Matching seeded jobs exist.

Test Data:

- Valid proposals generated from `[CONTROLLED_IMAGE_FIXTURE]`.

Steps:

1. Process the controlled image.
2. Review the generated proposals.
3. Select/accept the intended valid proposals.
4. Apply them to the job search.
5. Observe the resulting search criteria and job results.

Expected Result:

- Accepted proposals become active search criteria.
- The resulting job search uses the reviewed criteria.
- Matching seeded jobs are returned according to the deterministic job-search behavior.

Execution Date: 2026-08-21

Actual Result: The controlled image-search request was admitted, but image processing again terminated before any reviewed proposal was produced. Because the OCR pipeline failed with the existing `OCR_UNAVAILABLE` condition, the `TypeScript` proposal could not be generated or applied to the job search, so the expected controlled matching job could not be verified.

Status: Fail

Bug ID: BUG-IMG-02

IMG-04 — Edit an image-generated proposal before applying it

Type: Positive

Preconditions:

- Image processing has produced editable proposals.

Test Data:

- One generated proposal.
- A controlled replacement value.

Steps:

1. Process the controlled image.
2. Select one generated proposal.
3. Edit the proposal to the controlled replacement value.
4. Apply the reviewed proposals.
5. Observe the active search criteria.

Expected Result:

- The generated proposal can be edited before use.
- The edited value, rather than the original generated value, becomes the active criterion.
- The resulting job search reflects the user's reviewed value.

Execution Date: 2026-08-21

Actual Result: The valid image was admitted for processing, but the live image-search pipeline again terminated at the OCR stage with the existing `OCR_UNAVAILABLE` condition. No generated proposal was produced, so the proposal-editing step could not be performed.

Status: Fail

Bug ID: BUG-IMG-02

IMG-05 — Remove an image-generated proposal

Type: Positive

Preconditions:

- Image processing has produced multiple proposals.

Test Data:

- One generated proposal selected for removal.

Steps:

1. Process the controlled image.
2. Review generated proposals.
3. Remove one proposal.
4. Apply the remaining proposals.
5. Observe the active search criteria.

Expected Result:

- The user can remove an unwanted generated proposal before applying the search.
- The removed proposal does not become an active search criterion.
- Remaining accepted proposals continue to work normally.

Execution Date: 2026-08-21

Actual Result: The controlled multi-proposal image was admitted successfully, but the live image-search pipeline again failed at the OCR stage with the existing `OCR_UNAVAILABLE` condition. No validated set of image-generated proposals was produced, so the proposal-removal workflow could not be exercised.

Status: Fail

Bug ID: BUG-IMG-02

IMG-06 — Preserve a conflicting manually entered criterion

Type: Edge

Preconditions:

- A manual job-search criterion can be entered before image processing.
- The image fixture can produce a potentially conflicting proposal.

Test Data:

- Controlled manually entered criterion.

- `[CONTROLLED_IMAGE_FIXTURE]`.

Steps:

1. Enter the controlled manual search criterion.
2. Start the image-assisted search flow.
3. Process the controlled image.
4. Review the generated proposal that conflicts with the existing manual value.
5. Apply the reviewed image-generated criteria.
6. Observe how the conflict is handled.

Expected Result:

- Existing manual user input is not silently overwritten in violation of the implemented conflict policy.
- The conflict is handled predictably and remains under user control.
- The final active criterion reflects an explicit supported user decision.

Execution Date: 2026-08-21

Actual Result: The manual `Hanoi` location criterion was prepared successfully, but the subsequent image-processing request failed at the OCR stage with the existing `OCR_UNAVAILABLE` condition. No generated `Ho Chi Minh City` location proposal was produced, so the intended manual-versus-generated conflict behavior could not be exercised.

Status: Fail

Bug ID: BUG-IMG-02

IMG-07 — Reject an unsupported image file type

Type: Negative

Preconditions:

- The image-search interface is available.

Test Data:

- `[UNSUPPORTED_IMAGE]`

Steps:

1. Open the image-assisted search interface.
2. Provide required consent.
3. Attempt to select/upload `[UNSUPPORTED_IMAGE]`.

4. Observe validation behavior.

Expected Result:

- The unsupported file type is rejected.
- OCR/AI processing does not proceed as though the file were valid.
- The user receives an appropriate validation/error response.

Execution Date: 2026-08-21

Actual Result: The unsupported `text/plain` file was rejected by the image-search workflow. The application displayed the validation message "Choose one PNG or JPEG up to 5 MB." and did not proceed with OCR/AI image processing.

Status: Pass

Bug ID: —

IMG-08 — Reject an image exceeding the maximum upload size

Type: Boundary

Preconditions:

- The configured image upload-size limit is known.
- `[OVERSIZED_IMAGE]` exceeds that limit.

Test Data:

- Configured maximum size: `[MAX_IMAGE_SIZE]`
- Test image: size greater than `[MAX_IMAGE_SIZE]`.

Steps:

1. Open the image-assisted search interface.
2. Provide required consent.
3. Attempt to select/upload `[OVERSIZED_IMAGE]`.
4. Observe validation and processing behavior.

Expected Result:

- The image exceeding the configured maximum size is rejected.
- Resource-intensive OCR/AI processing does not proceed for an invalid oversized input.
- The user receives an appropriate file-size validation response.

Execution Date: 2026-08-21

Actual Result: The valid PNG image exceeded the configured 5 MB upload-size limit and was rejected by the image-search workflow. The application displayed the validation message "Choose one PNG or JPEG up to 5 MB." and OCR/AI processing did not proceed.

Status: Pass

Bug ID: —

IMG-09 — Cancel image processing

Type: Edge

Preconditions:

- Image-processing services are operational.
- The workflow exposes the supported cancellation behavior.

Test Data:

- `[CONTROLLED_IMAGE_FIXTURE]`

Steps:

1. Begin processing the controlled image.
2. Trigger the supported cancel action while processing is still active.
3. Observe the processing state.
4. Observe whether generated criteria are applied after cancellation.

Expected Result:

- The cancellation request is handled safely.
- The canceled processing operation does not unexpectedly apply unfinished generated criteria.
- The UI returns to a consistent usable state.

Execution Date: 2026-08-21

Actual Result: The active image-processing request was successfully cancelled using the `Cancel image search` action. Processing stopped, no unfinished generated criteria were applied, and the interface returned to a usable state.

Status: Pass

Bug ID: —

IMG-10 — Enforce image-search admission/rate limit

Type: Edge

Preconditions:

- Image-search admission control is operational.
- The configured local test admission/rate-limit threshold is known.
- An isolated browser/session can be used if required.

Test Data:

- `[CONTROLLED_IMAGE_FIXTURE]`
- Configured admission/rate-limit threshold: `[RATE_LIMIT_THRESHOLD]`

Steps:

1. Open the image-assisted search interface using the controlled session.
2. Perform valid image-processing requests up to the configured threshold.
3. Attempt another request that exceeds the allowed threshold/window.
4. Observe the application's admission-control behavior.
5. Verify that the application remains responsive after the rejected request.

Expected Result:

- Requests within the configured limit are admitted normally.
- A request exceeding the configured limit is rejected or delayed according to the implemented admission/rate-limit policy.
- The limit cannot be bypassed merely by repeatedly submitting through the same controlled context.
- The application remains stable after enforcing the limit.

Execution Date: 2026-08-21

Actual Result: The test began with 2 authenticated image-search admissions already used in the rolling one-hour window. Eight additional requests were admitted successfully, reaching the configured limit of 10 admissions. The next request was rejected with the `Image Search limit reached` condition, while the application remained responsive and usable.

Status: Pass

Bug ID: —

3. Bug Reports

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

3.1 Authentication Bugs

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

BUG-AUTH-06 — Unverified Candidate login does not direct the user to email verification

Related Test Case: AUTH-06

Description:
When a Candidate account in `PENDING_VERIFICATION` state attempts to log in using the correct password, SmartHire denies authentication but displays the same generic invalid-credentials response used for an incorrect password or non-existent email. The current authentication specification requires the response to direct the user toward email verification without exposing additional account information.

Preconditions:

- Candidate account exists.
- Account state is `PENDING_VERIFICATION`.
- Correct password is known.

Steps to Reproduce:

1. Open `http://localhost:3001/login`.
2. Enter the email of the pending-verification Candidate.
3. Enter the correct password.
4. Submit the login form.
5. Observe the response.

Expected Result:
Authentication is denied, but the user is directed toward the email-verification flow without exposing additional account data.

Actual Result:
Authentication is denied and the generic message `Email or password is incorrect.` is displayed. No verification guidance, link, or redirect is provided.

Severity: Medium

Status: Open

BUG-AUTH-08 — Protected-route return destination is lost after login

Related Test Case: AUTH-08

Description:
When a signed-out user directly accesses the protected `/profile` route, the application correctly redirects to the login page with a `returnTo=/profile` destination. However, after successful authentication, the user is redirected to `/dashboard` instead of returning to `/profile`.

Preconditions:

- User is signed out.
- `/profile` is a protected route.
- A valid active and verified Candidate account is available.

Steps to Reproduce:

1. Sign out of SmartHire.
2. Navigate directly to `http://localhost:3001/profile`.
3. Observe the redirect to the login flow.
4. Enter valid Candidate credentials.
5. Complete login.
6. Observe the post-login destination.

Expected Result:
After successful login, the user is returned to the original safe internal destination `/profile`.

Actual Result:
After successful login, the user is redirected to `/dashboard`, and the original `/profile` return destination is lost.

Severity: Medium

Status: Open

3.2 Image Search Bugs

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

BUG-IMG-02 — Live image-search OCR pipeline becomes unavailable before proposal generation

Related Test Cases: IMG-02, IMG-03, IMG-04, IMG-05, IMG-06

Description:
Valid supported image-search fixtures are successfully admitted, scanned for malware, decoded, and normalized, but the live image-search worker repeatedly fails at the OCR stage with `OCR_UNAVAILABLE`. The failure occurs with materially different valid fixtures and prevents OCR text and image-generated search proposals from being produced.

Steps to Reproduce:

1. Open `http://localhost:3001/jobs`.
2. Open `Search jobs from an image`.
3. Provide the required image-processing consent.
4. Upload a valid supported image fixture such as `ocr-104.jpg` or `multi-proposal-minimal.jpg`.
5. Start image processing.
6. Wait for processing to complete.

Expected Result:
A clean accepted image is recognized by OCR within the supported processing flow and proceeds to image-search proposal generation.

Actual Result:
Admission, malware scanning, decoding, and normalization succeed, but the live OCR stage terminates with

`OCR_UNAVAILABLE`. AI interpretation and proposal validation are never reached. The UI falls back to manual search.

Severity: Medium

Status: Open

4. Test Summary

Performed by: Lưu Chí Hải & Nguyễn Minh Khôi | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

All 50 selected manual functional test cases were executed on the local SmartHire PA5 test environment. The table below summarizes the recorded results after merging Hải's AUTH/PROF/IMG execution with Khôi's JOB/APP execution.

Test Area	Performed by	Planned	Executed	Pass	Fail	Not Run
UC-AUTH-03 — Log In	Lưu Chí Hải	10	10	8	2	0
UC-PROF-01 — Manage Candidate Profile	Lưu Chí Hải	10	10	10	0	0
UC-JOB-01 — Browse, Search, and Filter Jobs	Nguyễn Minh Khôi	10	10	10	0	0
UC-APP-01 — Apply for a Job	Nguyễn Minh Khôi	10	10	10	0	0
Feature 005 US2 — Find Jobs from an Image Query (User Scenario)	Lưu Chí Hải	10	10	5	5	0
Total	**Lưu Chí Hải & Nguyễn Minh Khôi**	**50**	**50**	**43**	**7**	**0**

4.1 Defect Summary

Performed by: Lưu Chí Hải | *Reviewed by:* Nguyễn Gia Quốc Uy | *Edited by:* Lưu Chí Hải

Bug ID	Related Failed Test Cases	Severity	Status	Recorded by
BUG-AUTH-06	AUTH-06	Medium	Open	Lưu Chí Hải
BUG-AUTH-08	AUTH-08	Medium	Open	Lưu Chí Hải
BUG-IMG-02	IMG-02, IMG-03, IMG-04, IMG-05, IMG-06	Medium	Open	Lưu Chí Hải

Overall execution result: 50/50 planned cases were executed. 43 passed and 7 failed. Every recorded failed test case is linked to an Open bug report. No JOB or APP defects were recorded because all 20 cases executed by Nguyễn Minh Khôi passed.